

생성 패턴

⚙️ 생성 패턴

객체의 생성과 조합을 캡슐화하여 특정 객체가 생성되거나 변경되어도 구조에 크게 영향을 미치지 않도록 하기 위한 패턴

🏭 팩토리 메소드 패턴(Factory Method Pattern) ★

객체를 생성할 때 `new` 키워드를 직접 쓰다 보면 코드가 뻘뻘해집니다. 유연한 코드를 위해 "**객체를 만들어내는 공장을 만드는 패턴**", 즉 팩토리 메서드 패턴을 알아봅시다.

"객체를 생성하는 인터페이스는 미리 정의하되, 실질적으로 어떤 클래스의 인스턴스를 만들지는 서브 클래스(자식)가 결정하게 하는 패턴"

쉽게 말해 **객체 생성(instantiation)의 역할을 서브 클래스에게 위임하는 것**입니다.

- **슈퍼 클래스(부모):** "여기서 객체가 생성되어야 해"라는 ****구조(Framework)****와 시점만 결정합니다.
- **서브 클래스(자식):** "구체적으로 어떤 객체(Truck인지 Ship인지)를 만들 것인가"에 대한 ****내용(Implementation)****을 결정합니다.

■ 사용하는 이유 (Why?)

1. 무엇을 만들지 미리 알 수 없을 때 (유연성)

- 코드를 짤 때는 구체적으로 어떤 객체가 필요할지 예측할 수 없거나, 상황에 따라 다른 객체를 생성해야 할 때 유용합니다. 부모 클래스는 구체적인 클래스 이름(`new Dog()`, `new Cat()`)을 몰라도 됩니다.

2. 객체 생성의 책임을 분리하고 싶을 때 (SRP 준수)

- 객체를 생성하는 복잡한 과정을 메인 비즈니스 로직에서 분리하여, **생성 전담 서브 클래스**에 몰아넣을 수 있습니다. 이렇게 하면 코드 관리가 편해집니다.

3. 구체적인 정보를 숨기고 싶을 때 (정보 은닉)

- 클라이언트(사용자)는 구체적으로 어떤 하위 클래스가 생성되는지 알 필요가 없습니다. 그저 팩토리 메서드가 던져주는 객체(`Animal`)를 받아서 쓰기만 하면 됩니다.
- 예제코드

```
public abstract class Product {
    public abstract void use();
}

public abstract class Factory{ //공장의 구조 설정
    public final Product create(String owner){
        Product p = createProduct(shape);
        return p;
    }
}
```

```
        protected abstract Product createProduct(String shape);
    }

    public class DriverFactory extends Factory { //구체적인 내용을 만드는 서브클래스

        @Override
        protected Product createProduct(String shape) {
            return new Driver(shape);
        }

    }

    public class Driver extends Product {

        private String shape;

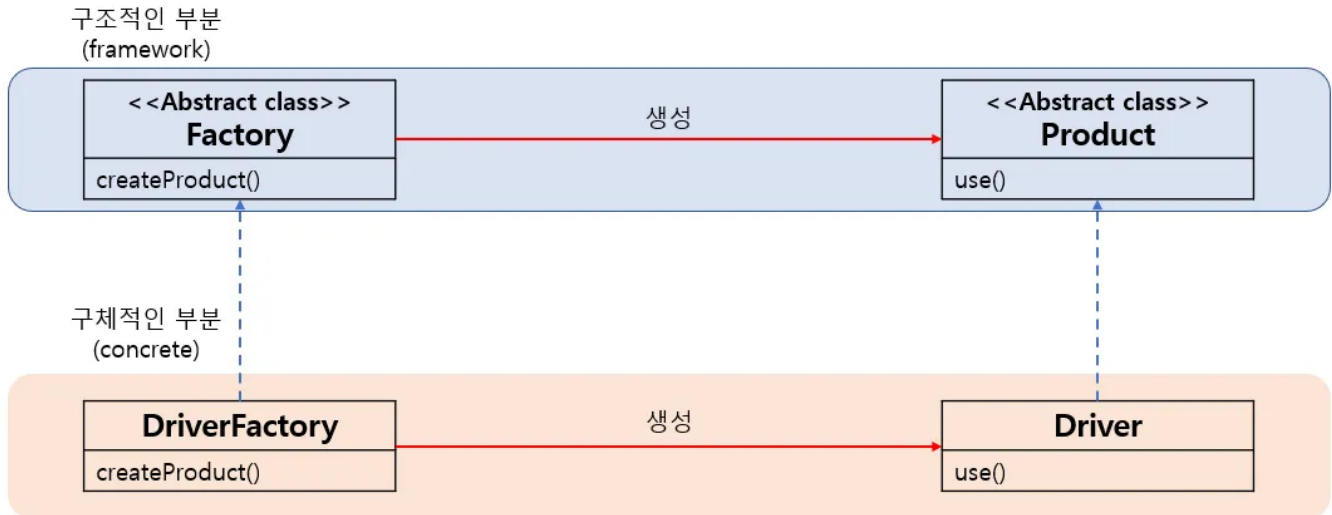
        public Driver(String shape) {
            System.out.println(shape + "드라이버를 만듭니다.");
            this.shape = shape;
        }

        @Override
        public void use() {
            System.out.println(shape + "드라이버를 사용합니다.");
        }

    }

    public class Main {
        public static void main(String[] args) {
            Factory factory = new DriverFactory();
            Product driver1 = factory.create("십자");
            Product driver2 = factory.create("일자");
            driver1.use();
            driver2.use();
        }
    }
}
```

- 예제 코드 UML



- 장점
 - 다른 제품을 만들때 구조적인 부분의 변화가 없음
 - OCP원칙을 지킬 수 있다
 - 단점
 - 상속을 사용하기 때문에 상속의 단점을 그대로 가짐
- 상속의 단점

1) 캡슐화 위반 (Breaking Encapsulation)

상속은 부모 클래스의 내부 구현을 자식 클래스에게 노출시킵니다.

- 자식 클래스가 부모 클래스의 ****내부 구현 상세(private이 아닌 부분들)****에 의존하게 됩니다.
- 부모가 어떻게 동작하는지 낱알이 알아야 제대로 상속받을 수 있으므로, 캡슐화(정보 은닉) 원칙이 깨집니다.

2) 깨지기 쉬운 상위 클래스 문제 (Fragile Base Class)

부모 클래스의 작은 변경이 자식 클래스 전체에 예상치 못한 오류를 일으킬 수 있습니다.

- **예시:** 부모 클래스의 **메서드 A** 내부 로직을 성능 개선을 위해 살짝 바꿨는데, 이를 상속받아 오버라이딩하던 자식 클래스 **B, C, D**가 갑자기 오작동할 수 있습니다.
- 이 때문에 부모 클래스를 수정할 때마다 모든 자식 클래스를 검토해야 하는 유지보수의 악몽이 시작됩니다.

3) 유연성 부족 (Inflexibility)

상속 관계는 **컴파일 시점**에 결정되어 고정됩니다.

- 실행(Runtime) 중에 부모 객체를 다른 것으로 교체하거나 행동을 바꿀 수 없습니다.
- 반면, ****합성(Composition)****을 사용하면 실행 중에 부품을 갈아끼우듯 동작을 바꿀 수 있습니다.

4) 클래스 폭발 (Class Explosion)

기능의 다양한 조합이 필요할 때, 상속을 쓰면 클래스 수가 기하급수적으로 늘어납니다.

- 예: 암호화된 파일, 압축된 파일, 암호화되고 압축된 파일 등을 모두 상속으로 구현하려면 `EncryptedFile`, `CompressedFile`, `EncryptedCompressedFile` 등 수많은 클래스를 만들어야 합니다.

- 많은 클래스를 정의해야 하기 때문에 **코드량이 증가**

"의존 역전 원칙 (DIP: Dependency Inversion Principle)이자, 여러분이 매일 쓰는 **DI(의존성 주입)**"

DI가 뭔데?

- 필요한 객체가 있으면 습관적으로 `new Dog()`, `new Cat()`을 코드 여기저기에 직접 씁니다.
- **질문:** "만약 `Dog` 클래스 이름이 `Puppy`로 바뀌면, 너는 `new Dog()` 라고 쓴 코드 100군데를 다 찾아서 고쳐야 해. 이게 객체지향적일까?"
- **해결:** "구체적인 클래스(`Dog`)에 의존하지 말고, 객체를 생성해 주는 공장(`AnimalFactory`) 인터페이스에 의존해라."

추상 팩토리 패턴(Abstract Factory Pattern)

"추상적인 공장에서 추상적인 부품들을 만들어내고, 이를 조합하여 제품을 완성하는 패턴"

이 패턴의 핵심은 *****구체적인 클래스에 의존하지 않고, 인터페이스(추상적 개념)만으로 부품을 조립한다*****는 점

- **클라이언트:** "나는 '버튼'과 '창'이 필요해." (구체적으로 맥 스타일인지 윈도우 스타일인지는 모름)
- **추상 팩토리:** "알겠어, 내가 알아서 세트로 맞춰줄게."

■ 사용하는 이유 (Why?)

1. "관련된 객체들"을 한 번에 묶어서 관리할 수 있다 (일관성 보장)

- 어떤 시스템이 여러 개의 부품(예: 버튼, 체크박스, 스크롤바)으로 구성되어 있고, 이들이 ****반드시 같은 테마(스타일)****여야 할 때 유용합니다.
- 구체적인 클래스(`SamsungButton`, `SamsungScreen`)에 직접 의존하지 않으므로, 나중에 테마를 통째로 바꿔야 할 때 공장만 교체하면 됩니다.

2. 객체 생성 코드를 클라이언트에서 분리한다 (결합도 감소)

- 사용자(Client) 코드는 `new ConcreteProduct()` 같은 구체적인 생성 코드를 전혀 몰라도 됩니다.
- 오직 추상적인 인터페이스(`createButton()`)만 바라보고 작업하므로, 생성 로직이 바뀌어도 클라이언트 코드는 수정할 필요가 없습니다.

3. 부품의 교체가 쉽다

- 인터페이스만 사용하여 코드를 작성했기 때문에, 실제 공장(Factory 구현체)만 갈아 끼우면 만들어지는 모든 부품이 한순간에 바뀝니다. (예: 윈도우 테마 → 맥 테마로 즉시 변경)
- 예제 코드

```
///추상적인 부품 A,B,C///
public interface A {
    var a1;
```

```
    var a2;
}

public interface B {
    var b1;
    var b2;
}

public interface C {
    var c1;
    var c2;
}

///추상적인 팩토리///
public interface Factory {
    public A createA();
    public B createB();
    public C createC();
}

///실제 부품 A, B, C///
public class RealA implements A{
    var a1;
    var a2;

    //A코드
}

public class RealB implements B{
    var b1;
    var b2;

    //B코드
}

public class RealC implements C{
    var c1;
    var c2;

    //C코드
}

///실제 팩토리///
public class RealFactory implements Factory{
    public A createA(){
        //A생성코드
    }

    public B createB(){
        //B생성코드
    }

    public C createC(){
```

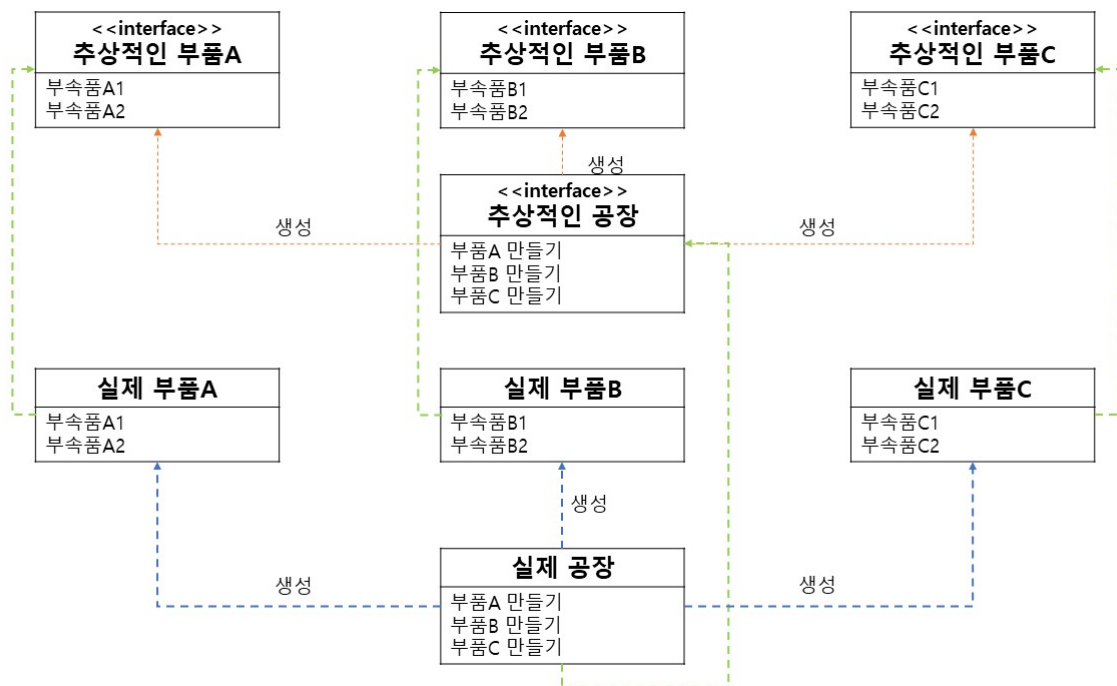
```

//C생성코드
}
}

RealFactory f = new RealFactory(); //실제 팩토리 생성
A creA = f.createA(); //실제 부품A 만들기
B creB = f.createB(); //실제 부품B 만들기
C creC = f.createC(); //실제 부품C 만들기

```

• 예제코드 UML



• 장점

1. 확장이 용이하다 (OCP 준수)

- 새로운 스타일이나 브랜드(구체적인 공장)를 추가하는 것이 매우 간단합니다.
- 예를 들어, **삼성** 공장만 있다가 **애플** 공장을 새로 만든다고 가정해 봅시다. 기존의 추상 공장 (인터페이스) 코드는 단 한 줄도 수정할 필요 없이, 그저 새로운 클래스(**AppleFactory**)만 하나 더 만들면 됩니다.
- 핵심: "공장을 아무리 많이 늘려도, 뼈대(인터페이스)는 건드리지 않습니다."

2. 구체적인 내용을 감출 수 있다 (캡슐화 & 느슨한 결합)

- 사용자(Client)는 이 공장이 내부적으로 어떤 부품(클래스 인스턴스)을 조립하는지 알 필요가 없습니다. 그저 "스마트폰 하나 주세요"라고 요청하면 알아서 삼성 폰이나 아이폰이 나옵니다.
- 덕분에 클라이언트 코드와 구체적인 클래스 간의 **결합도(Coupling)**가 낮아져 코드가 유연해집니다.

• 단점

1. 제품군을 확장하기 어렵다

- 가장 치명적인 단점입니다. 기존에 **스마트폰**과 **이어폰**만 만들던 공장들에게 갑자기 ****스마트 워치****라는 새로운 부품도 만들라고 시킨다면 어떻게 될까요?

2. 모든 공장이 공사 중 (수정의 파급 효과)

- 추상 공장 (인터페이스)**에 `createSmartWatch()` 메소드를 추가하는 순간, 이를 상속받은 ****모든 실제 공장들(삼성, 애플, 샤오미 등)****이 전부 에러를 뿜어냅니다. 모든 공장 클래스에 가서 일일이 `createSmartWatch()`를 구현해줘야 하기 때문입니다.
- 유지보수 비용 폭발:**

"만약 실제 공장이 10개인데 부품 2개를 추가해야 한다면? 최소 20번의 파일을 열어서 수정해야 하는 노가다 작업이 기다리고 있습니다."

■ 팩토리 메소드 패턴 vs 추상 팩토리 패턴

1. 추상화의 범위 (어떻게 만드는가?)

- 팩토리 메서드 패턴: "메소드(행동)를 추상화한다"**
 - 방식:** 상속(Inheritance)을 사용합니다.
 - 설명:** 부모 클래스에서 "객체를 만드는 `create()` 메소드"의 겉데기만 만들어두고, 자식 클래스에서 그 메소드를 오버라이딩하여 구체적인 객체를 만듭니다.
 - 핵심:** "만드는 ****방법(Method)****만 비워둘 테니, 자식이 채워 넣어라."
- 추상 팩토리 패턴: "팩토리(객체) 자체를 추상화한다"**
 - 방식:** 구성(Composition)을 사용합니다.
 - 설명:** 여러 개의 생성 메소드가 묶여 있는 "공장 클래스 자체"를 인터페이스로 만듭니다. 클라이언트는 이 공장 객체를 갈아 끼움으로써 생성되는 객체들을 통째로 바꿉니다.
 - 핵심:** "만드는 **공장(Factory)** 자체를 추상화해서 통째로 전달한다."

2. 사용하는 곳 (무엇을 만드는가?)

- 팩토리 메서드 패턴: "한 가지 제품의 다양한 버전"**
 - 사용처:** ****단일 제품(Single Product)****을 만들 때 사용합니다.
 - 예시:** **피자**를 만드는데, **치즈 피자**, **페퍼로니 피자**, **불고기 피자** 중 하나를 선택해서 만들어야 할 때.
 - 특징:** 부품 하나하나를 다양한 종류로 만들어 조립할 때 유용합니다.
- 추상 팩토리 패턴: "연관된 제품들의 묶음 (Family)"**
 - 사용처:** ****관련된 제품군(Product Family)****을 일관성 있게 생성해야 할 때 사용합니다.
 - 예시:** **피자 세트**를 만드는데, **도우+소스+토핑**이 ****반드시 같은 스타일(이탈리아식 or 미국식)****로 짝이 맞아야 할 때.
 - 특징:** 부품들이 서로 섞이면 안 되고, 일관된 테마를 유지해야 할 때 유용합니다.

구분	팩토리 메서드 패턴	추상 팩토리 패턴
핵심	상속으로 객체 생성 (Class)	객체 주입으로 객체 생성 (Object)

구분	팩토리 메서드 패턴	추상 팩토리 패턴
비유	단품 메뉴 주문하기	
(짜장면 vs 짬뽕)	세트 메뉴 주문하기	
(맥모닝 세트 vs 빅맥 세트)		
관점	"이 메소드를 구현해서 만들어"	"이 공장(객체)을 써서 만들어"

빌더 패턴(Builder Pattern) ★

객체를 생성할 때 생성자(Constructor)만으로는 해결하기 힘든 복잡한 상황이 올 때가 있습니다. 이때 구원투수로 등장하는 것이 바로 빌더 패턴입니다.

복잡한 객체의 생성 과정과 표현 방법을 분리하여, 동일한 생성 절차에서 서로 다른 표현 결과를 만들 수 있게 하는 패턴

■ 언제 사용하나요? (Why?)

1. 생성자의 매개변수가 너무 많을 때 (가장 큰 이유)

- 매개변수가 10개인 생성자를 상상해 보세요. `new User("홍길동", null, 20, null, "서울", ...)` 처럼 작성하다 보면, 몇 번째 인자가 나이이고 몇 번째가 주소인지 헷갈리게 됩니다.

2. 필수 인자와 선택 인자가 섞여 있을 때

- 어떤 값은 꼭 넣어야 하고, 어떤 값은 안 넣어도 될 때, 생성자로 처리하려면 수많은 오버로딩(Overloading) 생성자를 만들어야 합니다. 빌더는 필요한 것만 골라서 설정할 수 있습니다.
- 예제코드

```
public class Pizza {
    // final을 사용하여 값이 한번 초기화되면 변경되지 않도록 함
    private final String name;
    private final String dough;
    private final String edge;
    private final int price;

    public class Builder{ //생성자 대신 Builder 사용
        private final String name;
        private final String dough;
        private final String edge;
        private final int price;

        public Builder name(String name) {
            this.name = name;
            return this;
        }

        public Builder dough(String dough) {
            this.dough = dough;
        }
    }
}
```



```
        return this;
    }

    public Builder edge(String edge) {
        this.edge = edge;
        return this;
    }

    public Builder price(int price) {
        this.price = price;
        return this;
    }

    public Pizza build() {
        return new Pizza(this);
    }
}

public Pizza(Builder builder){
    this.name = builder.name;
    this.dough = builder.dough;
    this.edge = builder.edge;
    this.price = builder.price;
}

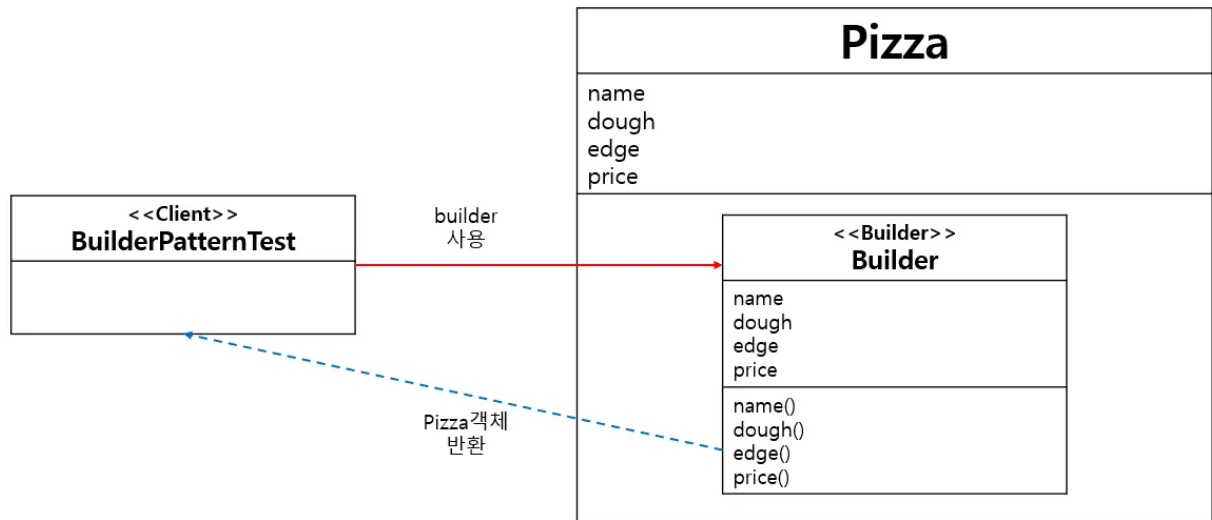
}

public class BuilderPatternTest{
    public static void main(String[] args) {

        //피자 빌드하기
        Pizza pizza = new Pizza.Builder()
            .name("콤비네이션피자");
            .dough("오리지널");
            .edge("체다치즈크러스트");
            .price(25000);
            .build();

    }
}
```

- 예제코드 UML



- 장점

1. 필요한 데이터만 쓱쓱 설정 가능 (유연성)

- 생성자를 쓰면 사용하지 않는 값에 `null`이나 `0`을 억지로 채워 넣어야 했지만, 빌더는 필요한 메소드만 호출하면 됩니다.

2. 코드가 소설처럼 읽힌다 (가독성)

- 코드를 보는 순간 각 값이 무엇을 의미하는지 명확해집니다.
 - **Before:** `new Pizza("Thin", "L", true);` (true가 뭐지?)
 - **After:** `Pizza.builder().dough("Thin").size("L").cheese(true).build();` (아, 치즈 추가구나!)

3. 객체를 '불변(Immutable)'으로 만들 수 있다 (안전성)

- 이게 아주 중요합니다. `Setter` 메소드를 열어두면 객체가 어디서 변할지 몰라 불안하지만, 빌더를 쓰면 객체 생성 시점에 값을 딱 한 번만 넣고 **수정자(Setter)를 아예 없앨 수 있습니다.** 변경 가능성을 최소화 하여 버그를 줄입니다.

- 단점

1. 코드량이 늘어난다 (번거로움)

- 빌더 클래스를 따로 만들어야 하므로 코드의 양이 늘어납니다. (다행히 실무에서는 **Lombok**의 `@Builder` 어노테이션이 이 문제를 해결해 줍니다.)

2. 구조가 바뀌면 공사 범위가 크다 (결합도)

- 객체의 필드가 이름에서 닉네임으로 바뀌면, 빌더 클래스도 수정해야 하고 빌더를 사용하는 모든 코드도 수정해야 합니다. 즉, 객체와 빌더 간의 결합도가 높습니다.

프로토타입 패턴(Prototype Pattern)

대부분의 객체는 **new** 키워드로 생성합니다. 하지만 어떤 객체는 생성하는 과정이 너무 복잡하거나 시간이 오래 걸릴 수 있습니다. 이때 사용하는 것이 바로 **프로토타입 패턴**입니다.

"기존의 인스턴스(원형, Prototype)를 복제(Clone)하여 새로운 인스턴스를 만드는 방법"

쉽게 말해 **복사기**와 같습니다. 처음 한 장은 힘들게 타이핑해서 문서를 만들지만(객체 초기화), 두 번째 장부터는 복사기 버튼만 누르면(clone) 똑같은 문서가 순식간에 나옵니다.

■ 언제 사용하나요? (Why?)

1. 객체를 생성하는 비용(Cost)이 너무 클 때 (성능 최적화)

- **상황:** 객체 하나를 만들 때마다 DB에서 엄청나게 많은 데이터를 가져와야 한다면?
- **해결:** **new**로 100번 생성해서 DB를 100번 조회하는 대신, **한 번만 DB에서 조회해서 완벽한 객체를 만들고, 나머지 99개는 그 객체를 메모리 상에서 복제해서** 씁니다. I/O 비용을 획기적으로 줄일 수 있습니다.

2. 생성 과정이 너무 복잡할 때

- **상황:** 생성자의 파라미터가 20개나 되고, 각종 설정 세팅이 복잡한 클래스가 있습니다.
- **해결:** 이미 세팅이 끝난 '샘플 객체'를 하나 만들어두고, 필요할 때마다 이를 복제해서 일부 값만 수정해 사용하는 것이 훨씬 편합니다.

3. 클래스로부터 인스턴스 생성이 어려운 경우

- 취급하는 오브젝트의 종류가 너무 많아서, 각각을 별도의 클래스로 만들기 부담스러울 때 원형(Prototype)을 복제하여 종류를 늘려나가는 방식을 사용합니다.
- 예제코드 - (출처 : <https://readystory.tistory.com/122>)

```
//java에서는 clone으로 프로토타입 패턴 구현이 가능  
//복사할 클래스 객체는 Cloneable인터페이스를 상속
```

```
public class Employees implements Cloneable {  
    private List<String> empList;  
  
    public Employees() {  
        empList = new Array<String>();  
    }  
  
    public Employees(List<String> list){  
        this.empList = list;  
    }  
  
    public void loadData(){  
        //데이터베이스에서 모든 직원 정보 복사  
        empList.add("James");  
        empList.add("Edward");  
        empList.add("David");  
        empList.add("Lisa");  
    }  
  
    public List<String> getEmpList() {
```

```

        return empList;
    }

    @Override
    public Object clone() throws CloneNotSupportedException{
        List<String> temp = new ArrayList<String>();
        for(String s : this.empList){
            temp.add(s);
        }
        return new Employees(temp);
    }
}

public class PrototypePatternTest{
    public static void main(String[] args) {
        Employees emps = new Employees();
        emps.loadData();

        //clone()메소드로 직원 정보 얻어내기
        Employees empsNew = (Employees) emps.clone();
        Employees empsNew1 = (Employees) emps.clone();
        List<String> list = empsNew.getEmpList();
        list.add("John");
        List<String> list1 = empsNew1.getEmpList();
        list1.remove("Lisa");

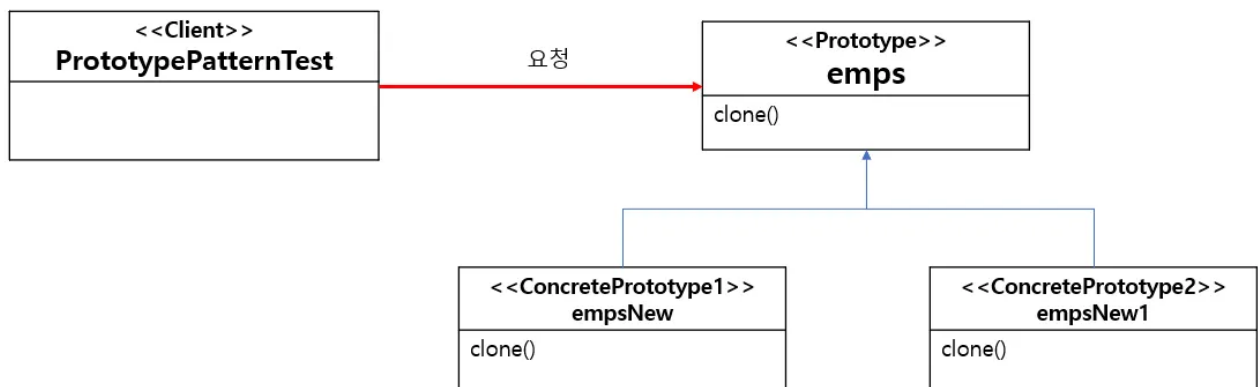
        System.out.println("emps List: "+emps.getEmpList());
        System.out.println("empsNew List: "+list);
        System.out.println("empsNew1 List: "+list1);

    }
}

//출력
emps List: [James, Edward, David, Lisa]
empsNew List: [James, Edward, David, Lisa, John]
empsNew1 List: [James, Edward, David]

```

- 예제 코드 UML



- 장점

1. 복잡한 객체 생성 과정을 숨길 수 있다 (캡슐화)

- 클라이언트는 복잡한 초기화 과정(DB 연결, 데이터 파싱 등)을 알 필요가 없습니다. 그저 `clone()` 메소드 하나만 호출하면 완성된 객체가 똑딱 나옵니다.

2. 객체 생성 비용을 획기적으로 줄인다 (성능)

- `new` 키워드로 객체를 새로 만드는 것보다, 메모리에 있는 데이터를 비트 단위로 복사하는 것이 훨씬 빠릅니다.
- 특히 **DB 접근**이나 **외부 API 호출**처럼 시간이 오래 걸리는 작업이 포함된 객체라면, 프로토타입 패턴은 선택이 아닌 필수일 수 있습니다.

3. 구체적인 클래스에 의존하지 않는다 (유연성)

- 클라이언트 코드는 구체적인 클래스(`ConcretePrototype`)를 몰라도 됩니다. 추상적인 인터페이스(`Prototype`)의 `clone()`만 호출하면 되므로 코드가 유연해집니다.
- 단점

1. `clone()` 구현 자체가 까다롭다 (가장 큰 문제)

- 단순히 `clone()`을 호출한다고 끝나는 것이 아닙니다. 객체 안에 **또 다른 객체**가 포함되어 있다면(예: `ArrayList`를 필드로 가진 경우), 기본 복사는 참조값만 복사하는 ****얕은 복사(Shallow Copy)****가 일어납니다.
- 원본과 복제본이 내부 데이터를 공유하게 되어, 복제본을 수정했는데 원본이 같이 바뀌는 치명적인 버그가 생길 수 있습니다.

2. 깊은 복사(Deep Copy)를 직접 구현해야 한다

- 위 문제를 해결하려면 내부 객체까지 일일이 새로 생성해서 넣어주는 **'깊은 복사'** 로직을 개발자가 직접 짜야 합니다.
- 객체의 구조가 복잡하거나 순환 참조(A가 B를 가지고, B가 다시 A를 가지는 경우)가 있다면 이 로직은 매우 복잡해집니다.

"DB에서 데이터를 읽어오는 비용이 너무 비싸서, **'캐싱(Caching)'** 개념으로 객체를 메모리에 두고 복사해서 쓰고 싶을 때 사용

🔒 싱글톤 패턴 ★

어떤 객체는 프로그램 전체에서 ****단 하나****만 있어야 할 때가 있습니다. 회사에 사장님이 두 명이면 곤란하듯이 말이죠. 이를 보장하는 것이 바로 **싱글톤 패턴**입니다.

"클래스의 인스턴스가 딱 1개만 생성되는 것을 보장하고, 어디서든 그 인스턴스에 접근할 수 있도록 하는 패턴"

■ 언제 사용하나요? (Use Case)

1. 리소스를 공유해야 할 때 (효율성)

- DB 연결 풀(Connection Pool), 스레드 풀(Thread Pool)처럼 생성 비용이 비싼 객체를 여러 번 만들지 않고 하나만 만들어 공유해서 쓸 때 사용합니다.

2. 전역적인 상태를 관리해야 할 때 (일관성)

- 환경 설정(Configuration) 관리자나 로그 기록 객체(Logger)처럼, 프로그램 어디서든 **똑같은 설정 값**을 보고 똑같은 파일에 기록해야 할 때 유용합니다.
- 예제코드

```
public class Singleton {
    private static Singleton singleton = new Singleton();

    private Singleton() {
        System.out.println("created Singleton");
    }
    public static Singleton getInstance(){
        return singleton;
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println("start");
        Singleton instance1 = Singleton.getInstance();
        Singleton instance2 = Singleton.getInstance();

        System.out.println("instance1과 instance2 동일성 비교::"+
(instance1==instance2)); // true
        System.out.println("end");
    }
}
```

- 예제코드 UML

Singleton
<code>singleton</code>
<code>Singleton() getInstance()</code>

- **장점: 효율과 공유**

1. 메모리 낭비를 방지한다 (메모리 효율)

- 객체를 **new**로 100번 생성하면 메모리 100칸을 차지하지만, 싱글톤은 최초 한 번만 생성하고 이를 계속 재사용하므로 메모리를 아낄 수 있습니다.

2. 데이터 공유가 쉽다 (전역 접근)

- 싱글톤 인스턴스는 전역(static)으로 선언되므로, 다른 클래스들 간에 데이터를 공유하기가 매우 쉽습니다.

- **단점: 치명적인 부작용 (Anti-Pattern의 위험)**

1. 테스트하기가 너무 힘들다 (가장 큰 단점)

- 단위 테스트는 서로 독립적이어야 하는데, 싱글톤은 자원을 공유하므로 테스트 순서에 따라 결과가 달라질 수 있습니다. 또한, **private** 생성자 때문에 가짜 객체(Mock)를 주입하기도 어렵습니다.

2. 객체 지향적이지 않다 (OCP 위배)

- 클라이언트 코드가 구체적인 싱글톤 클래스에 직접 의존하게 되어 결합도(Coupling)가 높아집니다.
- **private** 생성자를 쓰기 때문에 ****상속(Inheritance)****이 불가능하여 다형성을 활용할 수 없습니다.

3. 동시성 문제 (Thread Safety)

- 멀티스레드 환경에서 여러 스레드가 동시에 싱글톤 객체에 접근하여 상태를 바꾸려 하면, 데이터가 꼬이는 치명적인 버그(Race Condition)가 발생할 수 있습니다.

스프링 프레임워크는 이 싱글톤의 단점(테스트 어려움, 상속 불가 등)을 해결하면서 장점(메모리 효율)만 취하기 위해 ****싱글톤 레지스트리****라는 기능을 제공합니다.

- **자바의 싱글톤:** 개발자가 직접 **private** 생성자 만들고 **static** 변수 만들고 고생해야 함. (단점 많음)
- **스프링의 싱글톤:** 개발자는 평범한 클래스를 만들어도, **스프링 컨테이너가 알아서 객체를 딱 하나만 만들어서 관리해 줌.** (테스트도 쉽고, 상속도 가능!)

"그래서 스프링 빈(Bean)은 기본적으로 모두 싱글톤입니다."